

CS-341L - Fall 2025

Cache Lab: Part B - comparing human and AI generated code More Rubric details and hints on how to write the report

Work done in teams of two

Due date for Part B: December 8, 2025 at 11:59 pm

1 Extended Rubric

This document provides a lot more details about the grading rubric with which we will evaluate the work done for this project. It should also help understand what is expected from the teams and what to include on each portion of the report. For convenience the information in sections 1.2 and ?? are repeated here.

+10 pts: The submission is complete and well organized according to the specifications given in the statement of the project.

+ 26 pts: The performance evaluation of your human-generated code. Use what is provided with the project handout to test the performance.

+ 24 pts: The quality of your report according to the following criteria: organization, clarity, presentation of the results, completeness.

+ 40 pts: The quality of your analysis of results and conclusions.

The 24 points for the **quality of the report** include six points for each of the aspects indicated.

- **Organization:** follows the suggested report structure; you may change the names of the sections but it must include all the components, in particular, methods, data collected (the results) and the discussion of those results are essential. None of those could be missing.
- **Clarity:** of the introduction, conclusions, methods and how data was collected.
- **Presentation of the results:** substantial results must be presented and organized in a meaningful way. The organization should easily lead to their discussion.
- **Completeness:** all the components are included and with substantial content.

The 40 points for the **quality of the analysis of results and conclusions** are distributed among the following, roughly 10 points each.

- Discussion of the results: what did you get from the human-generated code; how did you produce that code; which aspects and specific approaches were critical to getting a good performance, also what did not work; if you tried something and didn't work, explain that too. Regarding the AI-generated code, discuss what made a difference in the prompts, in the refinement, etc.
- Comparison: compare all the human-generated code with the corresponding AI-generated code, notice the differences in style, in readability (for a human reader), in other aspects that you think will be relevant for the discussion about pros and cons of each.
- Ideas pursued, reasoning about the code written and generated.
- Conclusions: about the gains in performance on human-generated code; what in the prompts helped produce a better optimization of the AI-generated code?

1.1 Suggested Report structure

You must produce a report with all the information described in the project statement, we are not repeating that here, but we give more details of what goes in each part of the report.

1. Title and Summary – short summary and main claim(s).
2. Introduction and Motivation – Every report needs to have an introduction of what it contains, how is it organized and also must summarize the work done. The final paragraph of an introduction is always a list of all the sections and a one line description of what is contained in each section of the report. (See the guide to writing technical reports posted on canvas with this.)
3. Methods – the problems, model used, prompting strategy (iterative prompts, a lot of information at once, others?). If you are going to give too much information to the chatbot at a time, it would be advisable to then try at least another chatbot with the same information to determine differences in the code generated, if any. Save all the prompts and the code on each attempt; it's better to have more than less results, if they don't provide a great contribution to the work, you may discard them later.
4. Data Collection – measure the performance with the tool provided with the lab, examine the code to analyze its quality. Describe the reasoning you followed to generate the human code, describe several attempts you might have tried. With the AI-generated code: save all the prompts, save the whole conversation with the chatbot that led to code that has good performance. Show all the attempts you made at generating code and then analyze the most successful ones in the next section about the discussion. You need to collect enough information to provide substantial discussions and give foundation to your conclusions.
5. Exploratory Analysis/Discussion – explore possible reasons behind the results, discuss the results, compare specifics about the human-generated code and AI-generated code, provide explanations about the differences in the codes.

6. Conclusion — what you learned, conclusions for using AI in code generation; these conclusions must be founded on the results obtained from this project and point to those results.
7. References – a list of all the sources used to perform the tasks for this assignment. Include all the kinds of help used, if any.
8. Appendices – If any, these may include: c code, log of prompts, intermediate results, etc.

Special restriction: please do not include screenshots in your report, they occupy a lot of space, make the pdf unmanageable, and are usually unreadable. The history of your interaction with the AI tools must be stored in plain text files with the .txt extension. The code must be stored in .c files.

1.2 AI Tool Policy

Your use of genAI tools for this assignment is subject to the following constraints:

- You may NOT use AI for your human-written code, it will violate the rules of the assignment and will get you a zero.
- For AI-generated code:
 - You may approach the AI however you like but must record protocol for each problem (the prompting style)
 - You MUST record at a minimum:
 - * Model name & version
 - * Exact prompts and follow-ups
 - * How the code evolved (story, not just the final answer)

2 What to submit

You will compress the following files into a tar file and submit the tar file on canvas at the place for this assignment:

Human generated code The `trans_human.c` file and the performance evaluation of this in the `trans_human_results.csv` file. You need to follow the instructions to evaluate the performance of this code.

AI generated code The best `ai_trans.c` file generated with AI tools and the `ai_trans_results.csv` with the performance evaluation of this code.

Report The report produced from the execution of the tasks described in the statement of the projects with the contents described there and extended here.

2.1 Performance Evaluation

Human and AI generated code for the optimized transpose function will be evaluated by you according to the following aspects.

The correctness and performance of your `transpose_submit` function on three different-sized output matrices:

- 32×32 ($M = 32, N = 32$)
- 64×64 ($M = 64, N = 64$)
- 61×67 ($M = 61, N = 67$)

For each matrix size, the performance of your `transpose_submit` function is evaluated by using `valgrind` to extract the address trace for your function, and then using the reference simulator to replay this trace on a cache with parameters ($s = 5, E = 1, b = 5$).

Your performance score for each matrix size scales linearly with the number of misses, m , up to some threshold:

- 32×32 : 8 points if $m < 300$, 0 points if $m > 600$
- 64×64 : 8 points if $m < 1,300$, 0 points if $m > 2,000$
- 61×67 : 10 points if $m < 2,000$, 0 points if $m > 3,000$

All code submitted must be correct to receive any performance points for a particular size. All code submitted only needs to be correct for these three cases and you can optimize it specifically for these three cases. In particular, it is perfectly OK for your function to explicitly check for the input sizes and implement separate code optimized for each case. It is probably easier to include the code for each matrix size in a single `.c` file for the human generated code and another single file for the ai-generated code.